PROGRAMAÇÃO OPEN BIM COM PYTHON

Leandro Brito e Marco Aurélio Souza Mangan

> **É impossível aprender algo que a gente já sabe.**
>
> *Tiago Mattos*

PUCRS online

uol edtech

Conheça o livro da disciplina

CONHEÇA SEUS PROFESSORES 3

Conheça os professores da disciplina.

EMENTA DA DISCIPLINA 4

Veja a descrição da ementa da disciplina.

BIBLIOGRAFIA DA DISCIPLINA 5

Veja as referências principais de leitura da disciplina.

O QUE COMPÕE O MAPA DA AULA? 6

Confira como funciona o mapa da aula.

MAPA DA AULA 7

Veja as principais ideias e ensinamentos vistos ao longo da aula.

RESUMO DA DISCIPLINA 22

Relembre os principais conceitos da disciplina.

AVALIAÇÃO 23

Veja as informações sobre o teste da disciplina.

Conheça seus professores



LEANDRO BRITO
Professor Convidado

Leandro Brito de Gouvêa é engenheiro civil formado pela Universidade Federal do Paraná (UFPR), com trajetória consolidada na aplicação e disseminação do Building Information Modeling (BIM) no Brasil. É sócio-fundador da CWBIM Engenharia, empresa especializada em soluções BIM para compatibilização de projetos, estimativas de custos, planejamento de obras e gestão da informação técnica em empreendimentos de diferentes portes. Entre 2015 e 2018, foi consultor técnico no Departamento de Gestão de Obras Públicas do Estado do Paraná, contribuindo para o plano estadual de fomento ao BIM. Possui experiência acadêmica como professor convidado na UTFPR e no IDD – Educação Avançada, atuando em cursos de especialização e pós-graduação com temas como gestão estratégica de projetos, colaboração interdisciplinar com BIM, Lean Construction e organização da informação digital na construção. Combinando prática profissional, pesquisa acadêmica e atuação como docente, dedica-se a promover a modernização dos processos construtivos, com foco em eficiência, rastreabilidade e qualidade.

MARCO AURÉLIO SOUZA MANGAN
Professor PUCRS



Marco Aurélio Souza Mangan possui graduação em Ciência da Computação pelo Instituto de Informática/UFRGS (1995), mestrado em Computação pelo Instituto de Informática/UFRGS (1998) e doutorado em Engenharia de Sistemas e Computação pelo PESC/UFRJ (2006). Atualmente é professor adjunto da Escola Politécnica/PUCRS. Tem experiência na área de Ciência da Computação, com ênfase em Linguagens de Programação, atuando principalmente nos seguintes temas: programação de computadores, engenharia de software, ambientes de desenvolvimento de software, CSCW e reutilização de software. Ministra regularmente cursos de extensão na área. Atua também na UNATI/PUCRS, no curso Vida Digital para Terceira Idade.

Ementa da Disciplina

Introdução ao uso de Python para automação e personalização de processos em BIM. Desenvolvimento de scripts para manipulação de dados BIM. Integração de Python com softwares BIM e formatos de dados abertos, como IFC. Aplicações práticas e estudos de caso na programação Open BIM.

Bibliografia da Disciplina

As publicações destacadas têm acesso gratuito.

Bibliografia básica

EASTMAN, C.; TEICHOLZ, P.; SACKS, R.; LISTON, K. BIM Handbook: A Guide to Building Information Modeling for Owners, Managers, Designers, Engineers, and Contractors. Third Edition. New Jersey: John Wiley & Sons, 2018.

LOTH, Andreas. Learn Python Visually: Creative Coding with Processing.py. No Starch Press, 2021.

LACEY, Andrew. Python Programming for Open BIM: Automating tasks in the AEC industry with IFC and IfcOpenShell. Packt Publishing, 2020.

Bibliografia complementar

DIMYADI, John. Python Scripting for Building Information Modeling (BIM) and Computer-Aided Design (CAD). Springer, 2019.

ZHANG, Jane. Practical Python for BIM: A Practical Guide to Applying Python in the Building Industry. 2022.

BRADBURY, Ryan. Python for BIM and Revit: Unlocking Advanced BIM Workflows. Wiley, 2021.

O que compõe o Mapa da Aula?

MAPA DA AULA

São os capítulos da aula, demarcam momentos importantes da disciplina, servindo como o norte para o seu aprendizado.

EXERCÍCIOS DE FIXAÇÃO

Questões objetivas que buscam reforçar pontos centrais da disciplina, aproximando você do conteúdo de forma prática e exercitando a reflexão sobre os temas discutidos. Na versão online, você pode clicar nas alternativas.

PALAVRAS-CHAVE

Conceituação de termos técnicos, expressões, siglas e palavras específicas do campo da disciplina citados durante a videoaula.

VÍDEOS

Assista novamente aos conteúdos expostos pelos professores em vídeo. Aqui você também poderá encontrar vídeos mencionados em sala de aula.

PERSONALIDADES

Apresentação de figuras públicas e profissionais de referência mencionados pelo(a) professor(a).

LEITURAS INDICADAS

A jornada de aprendizagem não termina ao fim de uma disciplina. Ela segue até onde a sua curiosidade alcança. Aqui você encontra uma lista de indicações de leitura. São artigos e livros sobre temas abordados em aula.

FUNDAMENTOS

Conteúdos essenciais sem os quais você pode ter dificuldade em compreender a matéria. Especialmente importante para alunos de outras áreas, ou que precisam relembrar assuntos e conceitos. Se você estiver por dentro dos conceitos básicos dessa disciplina, pode tranquilamente pular os fundamentos.

CURIOSIDADES

Fatos e informações que dizem respeito a conteúdos da disciplina.

DESTAQUES

Frases dos professores que resumem sua visão sobre um assunto ou situação.

ENTRETENIMENTO

Inserções de conteúdos para tornar a sua experiência mais agradável e significar o conhecimento da aula.

CASE

Neste item, você relembra o case analisado em aula pelo professor.

MOMENTO DINÂMICA

Aqui você encontra a descrição detalhada da dinâmica realizada pelo professor.

Mapa da Aula

Os tempos marcam os principais momentos das videoaulas.

AULA 1 • PARTE 1

Aspectos iniciais

01:08

O professor **Leandro** se apresenta como engenheiro civil e contextualiza sua trajetória desde 2012 no estudo do **BIM**. Define o objetivo do curso: programar em **Python** para manipular arquivos **IFC** (Open BIM). Destaca três eixos centrais — **Python**, **IFC** e **exemplos práticos** — que serão explorados. Explica como esses conhecimentos contribuem para **fluxos de trabalho** mais eficientes na construção civil, reduzindo **retrabalhos** e permitindo **automação** e **análise de dados**.

Nesta etapa são apresentados os conceitos de **BIM**, entendido como metodologia de organização e gestão da **informação da construção,** o **Open BIM,** que garante **interoperabilidade** entre softwares e facilita a colaboração, o **IFC** como formato **padrão ISO** de troca de dados, a **linguagem Python** como ferramenta **intuitiva** e acessível, e a importância dos **fluxos de trabalho** eficientes para reduzir **retrabalho** no processo de projeto e execução.

03:30

Preparação do ambiente

É detalhado o processo de **instalação do Python**, considerando diferentes sistemas operacionais (**Windows** e **Mac**). São explicadas as versões compatíveis com a biblioteca **IFC Open Shell** (3.12). O professor mostra como configurar a variável **Path**, garantindo funcionamento adequado. Em seguida, orienta a instalação do **VSCode**, descrevendo-o como **IDE** gratuita e eficiente. Também introduz o uso do **GitHub** para versionamento e armazenamento de código, explicando opções de **download em zip** ou criação de **forks**. O objetivo é preparar um ambiente integrado que suporte a programação e manipulação de modelos **BIM**.

Nesta parte, consolidam-se noções técnicas indispensáveis: a versão **Python 3.12** como requisito de compatibilidade, o papel do **Path** na execução correta do software, a função do **VSCode** como ambiente de programação, o **GitHub** como ferramenta de **colaboração e histórico de projetos,** e o uso do **IFC Open Shell** como biblioteca que conecta **Python** ao universo do **BIM**.

Primeiros conceitos em Python

14:13

A aula inicia a parte prática em **Python,** apresentando **variáveis, tipos de dados e operadores** como fundamentos da linguagem. O professor mostra exemplos de criação de variáveis **(nome, idade, altura)** e formas de impressão no **console** usando print. Explica diferenças entre **números inteiros, ponto flutuante, booleanos e strings,** associando-os ao conceito de **objetos** em programação. Faz uma analogia entre primitivos do **CAD** (linhas e arcos) e objetos do **BIM** (paredes, portas, janelas), mostrando como esses conceitos abstratos se aplicam na modelagem.

Nesta parte, são introduzidos os conceitos de **variáveis** como elementos básicos de armazenamento de dados, os **tipos de dados primitivos (inteiros, ponto flutuante, booleanos e strings)** que estruturam a informação em **Python**, a função print como recurso essencial para **visualização no console,** e a noção de **objetos** como estruturas que encapsulam **atributos** e **métodos**, fundamentais para compreender a correspondência entre a lógica da programação e a representação de elementos no **BIM**.

AULA 1 • PARTE 2

00:26

Variáveis e operadores

O professor introduz o conceito de variável booleana, explicando que só pode assumir valores verdadeiro ou falso, com atenção à sintaxe em Python. Em seguida, apresenta a variável texto, que deve ser escrita entre aspas. Mostra como declarar variáveis e utilizá-las no programa. A partir daí, inicia o tema operadores, separando-os em matemáticos, comparativos e lógicos. Demonstra operações de soma, subtração, multiplicação, divisão e módulo, explicando como o restante da divisão é calculado. Depois, mostra exemplos de comparação entre variáveis, destacando o retorno em valores booleanos, e diferencia atribuição de comparação. Por fim, introduz os operadores lógicos (and, or, not), ilustrando com o caso de uma porta que pertence a dois ambientes simultaneamente.

Variáveis **booleanas** são fundamentais para expressar condições em programação, representando apenas dois estados possíveis. **Operadores aritméticos** realizam cálculos básicos. **Operadores de comparação** retornam valores lógicos que podem ser utilizados em decisões. **Operadores lógicos** permitem combinar condições e são a base das estruturas de controle. A distinção entre **atribuição** (=) e **comparação** (==) é um pré-requisito essencial.

Estruturas de controle

08:52

O professor apresenta as **estruturas de controle,** começando pelos condicionais. Mostra a sintaxe do **if** em Python, utilizando a variável idade para verificar maioridade, e a estrutura **else** para lidar com condições alternativas. Demonstra que o bloco else não é obrigatório. Em seguida, introduz os laços de repetição, com destaque para o **for**, percorrendo elementos de uma **lista** (paredes, janelas, portas). Explica que o nome da variável dentro do loop pode ser definido livremente. Depois, mostra o laço **while**, que executa repetidamente até uma condição ser satisfeita, ressaltando a importância de atualizar a variável para evitar loops infinitos.

As **estruturas condicionais** permitem que o programa execute caminhos diferentes com base em expressões lógicas. Os **laços de repetição** automatizam tarefas que envolvem conjuntos ou listas de elementos, sendo indispensáveis em processamento de dados. O **for** é mais adequado quando se conhece previamente a coleção de elementos; o **while** é usado quando a condição de parada depende de cálculos ou eventos dinâmicos.

14:48

Funções e coleções

O professor introduz as **funções**, explicando a sintaxe com **def**, parâmetros, retornos e chamadas, demonstrando como reutilizar código. Em seguida, aborda as **coleções: listas,** que armazenam elementos ordenados; **dicionários**, organizados em pares chave–valor; **sets**, que não permitem repetição; e **tuplas**, imutáveis. Mostra exemplos práticos de uso, destacando as diferenças entre estruturas mutáveis e não mutáveis.

Funções encapsulam blocos de código e favorecem a **reutilização. Listas** permitem armazenar sequências ordenadas e modificáveis. **Dicionários** estruturam dados por **chave e valor. Sets** garantem **unicidade** de elementos. **Tuplas** asseguram **imutabilidade**, característica essencial em contextos que exigem consistência.

AULA 1 • PARTE 3

Estrutura do IFC

00:26

O **IFC (Industry Foundation Classes)**, criado pela **BuildingSmart** e padronizado pela **ISO 16739**, assegura **interoperabilidade** entre softwares e agentes da construção. Divide-se em **esquema** (estrutura de dados) e **arquivo** (armazenamento de projetos). O **kernel** organiza premissas básicas, expandindo-se em extensões de **controle**, **produto** e **processos**. Acima, estão a camada de **interoperabilidade**, com elementos comuns, e os **domínios específicos**, como elétrica e hidráulica. O exemplo de busca por **IFCWall** ilustra a consulta às entidades.

O IFC é estruturado em **camadas hierárquicas:** núcleo (kernel), extensões, interoperabilidade e domínios específicos. Cada entidade segue a nomenclatura **IFC + sigla em inglês,** o que padroniza e evita ambiguidades computacionais.

05:24

Entidades e atributos

As entidades, como **IFCWall**, são definidas por **heranças** no esquema. Além de elementos de construção, abrangem anotações e controles. Cada objeto possui **atributos numerados** (ex.: **GlobalID**, **Name**, **Local Placement**) e **não numerados**, que registram relações com outros elementos. O professor mostra como buscar esses atributos no **arquivo IFC**, estruturado no formato **STEP**, usado em diferentes indústrias.

O IFC organiza entidades por **herança semântica,** associando atributos próprios e relacionais. O formato **STEP** garante legibilidade universal e intercâmbio seguro entre softwares.

09:08

Modificação e leitura em software

Demonstra-se a abertura de um modelo no **Solibri**, a busca de uma parede pelo **GUID** e a alteração do atributo **Name** direto no arquivo texto, confirmada no visualizador. Explica-se que edições manuais não são recomendadas, devendo-se usar o **IFC OpenShell** em **Python**, configurado em **ambiente virtual**. São mostrados os passos de instalação, importação, abertura de arquivos e consulta de entidades por **byGUID** e **byType**, além da listagem de elementos do projeto.

O **GUID** identifica unicamente cada entidade. O uso de bibliotecas como o **OpenShell** garante integridade do arquivo. Ambientes virtuais em **Python** isolam dependências e permitem manipulação segura.

AULA 1 • PARTE 4

00:26

Entidades no modelo BIM

O professor examina as **entidades do modelo BIM**, como proxy, columns, door, slab, wall e opening element. Mostra como identificar objetos classificados como **IFC Building Element Proxy,** usando comandos no **IFC File by type.** Em seguida, demonstra o uso do **print** para listar objetos, revelando que cada um possui ID, nome genérico e atributos de representação. Aponta a diferença entre **instâncias (objects)** e **tipos (types),** explicando que a relação entre eles é feita por funções de associação no esquema IFC. Ao consultar a documentação, destaca como atributos globais e específicos são herdados e como parâmetros identificam objetos e tipos. Finaliza mostrando a necessidade de acessar listas retornadas para identificar corretamente elementos, como tanque, armário, lavatório e vasos sanitários.

O **Building Element Proxy** serve como categoria genérica para elementos que não se encaixam em classificações específicas do IFC. A distinção entre **object** e **type** estrutura a organização dos modelos: objetos são instâncias concretas no projeto, enquanto os tipos trazem a informação normativa que os caracteriza. Relações no esquema IFC definem heranças e vínculos obrigatórios, como atributos globais e relation type.

Filtragem e categorização de objetos

07:21

O professor passa a detalhar como **filtrar objetos com torneiras.** Cria listas específicas para separar os elementos que possuem esse atributo, aplicando condicionais em **Python.** Demonstra a impressão de resultados, diferenciando tanques, armários, lavatórios e vasos sanitários. Introduz o uso de **dicionários** para associar cada objeto ao tipo de torneira correspondente, estabelecendo pares chave-valor. Em seguida, mostra como **quantificar os objetos** por tipo de torneira, utilizando a função update para somar ocorrências. O processo de codificação é mostrado passo a passo, com execução em loop e checagem de resultados.

A **classificação incorreta** de objetos no IFC mostra a relevância de convenções e da documentação. A integração entre **dados normativos** e **tratamento computacional** sustenta a confiabilidade das análises BIM, consolidando a importância do alinhamento entre modelagem e programação.

O uso de **listas** e **dicionários** em Python permite estruturar dados do modelo BIM, facilitando tanto a classificação quanto a contagem. O condicional **if** diferencia elementos conforme atributos, enquanto o **update** garante a atualização dinâmica dos valores. Essa abordagem exemplifica a integração prática entre lógica de programação e modelagem IFC.

14:16

Correções e recapitulando a aula

O professor identifica limitações de classificação no modelo, como a **bancada** que aparece sem torneira por não estar vinculada corretamente ao proxy. Para resolver, combina listas de diferentes tipos de entidades, como *sanitários* e *proxies,* e ajusta manualmente os dicionários para incluir os casos ausentes. Após validar as quantificações, ele apresenta o resultado final, que contabiliza tipos de torneiras por ambiente. Na sequência, realiza a **recapitulação da Aula 1,** retomando os principais aprendizados: interação entre **IFC** e **Python**, configuração do ambiente (Python, VSCode, IFC Open Shell), fundamentos básicos da linguagem Python, estrutura do **esquema IFC,** distinção entre esquema e arquivo, além da prática de manipulação do modelo BIM e da extração de informações como IDs e atributos.

EXERCÍCIO DE FIXAÇÃO

Qual operador em Python diferencia atribuição de comparação?

- =
- ==
- :=
- !=

EXERCÍCIO DE FIXAÇÃO

Qual estrutura é mais adequada para percorrer listas em Python?

- while
- for
- if
- def

Respostas desta página: alternativas 2, 2

AULA 2 • PARTE 1

Aspectos iniciais

01:08

A aula inicia com a proposta de desenvolver um **projeto avançado** em **Python**, integrando **IFC OpenShell** e **Streamlit**. O professor apresenta o Streamlit como ferramenta para criar **interfaces web** de forma simplificada, consultando a **documentação oficial** e organizando a instalação no **VSCode**. Demonstra a ativação do ambiente virtual, instalação via **pip install streamlit** e preparação do arquivo principal em **.py**, substituindo o uso de notebooks Jupyter. Introduz a prática de importar bibliotecas com apelidos (import streamlit as st) e a estrutura do bloco if __name__ == "__main__": para organização modular do código.

O **Streamlit** é uma biblioteca Python de código aberto que permite desenvolver aplicações interativas rapidamente. O padrão if __name__ == "__main__": garante que o código principal seja executado apenas quando o arquivo for chamado diretamente, evitando conflitos em projetos com múltiplos módulos.

03:34

Estruturação do projeto e integração inicial

O professor organiza o projeto criando a pasta **app** e o arquivo **app.py**, que passa a ser o ponto de entrada. Demonstra o uso de **importações** e da convenção **st.** para simplificar comandos. Ensina a rodar o Streamlit pelo terminal com streamlit run app/app.py, mostrando a geração automática de uma **interface web** acessível localmente. Destaca a importância de salvar alterações para atualização imediata da aplicação. Encerra com a perspectiva de hospedar a aplicação e compartilhar via **endereço IP**.

A execução pelo terminal é necessária porque o Streamlit funciona como servidor local. A atualização automática permite **prototipagem rápida.** O arquivo app.py é reconhecido como script Python central e deve ser organizado em uma estrutura modular desde o início.

Modularização e extração de dados IFC

07:30

O professor inicia a integração com o **IFC OpenShell**, criando o módulo **IFC_functions.py**. Define funções para leitura de arquivos, começando com get_paredes, que identifica e retorna entidades **IFCWall**. Explica a necessidade de usar caminhos relativos corretos e demonstra a exibição dos resultados no Streamlit. Em seguida, amplia a lógica para extrair **alvenarias** e **partes de elementos**, implementando a função alvenarias. Corrige erros de nomenclatura ao diferenciar função e variável, mostrando boas práticas de programação. O resultado é a listagem de **paredes** e **camadas construtivas** (como tijolos e acabamentos) diretamente na interface web.

O **IFC OpenShell** permite manipular modelos IFC via Python, acessando entidades com file.by_type(). A modularização em funções separadas (ex.: abrir_IFC, paredes, alvenarias) facilita a manutenção e reutilização. O retorno de entidades IFC no Streamlit ilustra a união entre **modelagem BIM** e **aplicações web**.

AULA 2 • PARTE 2

Filtro de alvenarias

00:26

O professor inicia a análise do modelo, destacando a necessidade de **filtrar elementos por material**. A partir do esquema IFC, ele elimina componentes que não utilizam **tijolo cerâmico**, criando um filtro em Python. O processo envolve identificar o atributo **Name** no schema, aplicar condição com if e organizar as partes em listas específicas de **alvenarias**, resultando em 25 partes de tijolo.

A filtragem por material no **IFC** garante padronização e consistência na análise, permitindo isolar elementos relevantes do projeto e eliminar informações irrelevantes.

01:27

Correções e recapitulando a aula

A partir das alvenarias filtradas, o professor ensina a localizar a **propriedade de área** no modelo. Explica que a consistência decorre do **plano de execução BIM**, que padroniza atributos em todos os projetos. Demonstra a navegação pelos **Property Sets** vinculados às entidades, utilizando a relação IsDefinedBy para acessar conjuntos de propriedades. Identifica o **Component Quantity** e o **Component Properties**. Com código em Python, busca a **área de componente projetada líquida**, exibindo valores como 8,47 m^2 e 2,712 m^2. O procedimento mostra como reutilizar funções para diferentes elementos (alvenarias, pisos, acabamentos).

O **Property Set** agrupa informações normativas e quantitativas de cada objeto IFC, assegurando interoperabilidade. O atributo **Element Quantity** define métricas como áreas e volumes, fundamentais para medições consistentes.

Estruturação e soma de áreas por material

14:27

O professor orienta a **vincular áreas calculadas aos respectivos materiais**. Explica a criação de dicionários em Python, associando o **nome do material** às áreas extraídas. Mostra como estruturar listas de áreas e iterar com for para consolidar dados. Demonstra a soma das áreas de **tijolo cerâmico**, obtendo valores agregados como 122,76 m^2. Introduz a função **round()** para limitar casas decimais e melhorar a legibilidade dos resultados. O processo é generalizado, permitindo a aplicação para diferentes materiais e tipologias.

O uso de **dicionários** e **loops** em Python organiza dados quantitativos de modo reutilizável, viabilizando análises comparativas entre materiais e simplificando o gerenciamento de informações em modelos BIM.

AULA 2 • PARTE 3

Lista de materiais e áreas

00:26

O professor inicia explicando como refatorar o código para listar **partes** e seus respectivos **materiais de construção** em vez de depender de nomes digitados manualmente. Ele cria funções para identificar automaticamente os materiais vinculados às partes e consolidar essa informação em um **dicionário**. A partir disso, gera uma tabela estruturada em **dataframe**, permitindo visualização clara das áreas associadas a cada material. O sistema ainda oferece exportação em **CSV**, destacando o ajuste necessário de **ponto para vírgula** devido à configuração regional.

A automatização na identificação de materiais evita erros manuais e garante consistência. O uso de **dataframes** organiza dados em formato tabular, facilitando análises e exportações.

09:02

Upload de arquivos IFC

É introduzida a funcionalidade de **upload de arquivos IFC** via Streamlit, substituindo a necessidade de caminhos fixos. O professor demonstra a integração do **st.file_uploader** e a adaptação das funções existentes para receber o arquivo como argumento. Aborda a diferença entre execução local e em nuvem, destacando o uso de **arquivos temporários** e manipulação em **binário** para garantir compatibilidade. Após ajustes, o sistema consegue carregar e processar arquivos enviados, conectando-os às funções de listagem de materiais.

O **upload dinâmico** amplia a aplicação para diferentes projetos IFC. O uso de **arquivos temporários (tempfile)** em Python garante portabilidade e leitura correta em servidores.

Exceções e projetos grandes

14:38

Testando arquivos maiores, surgem erros porque nem todas as partes possuem **área** definida (por exemplo, colunas com apenas volume). O professor resolve aplicando cláusulas **try/except**, que evitam travamentos quando atributos inexistem. Demonstra a abertura de projetos complexos, incluindo formatos **IFCZIP**, e reforça a robustez do código para gerar tabelas completas mesmo com inconsistências nos dados. Ao final, mostra a extração de quantidades e áreas de diversos **revestimentos** e **materiais** em um edifício de grande porte.

O tratamento de exceções é essencial em **processamento de dados de construção** para lidar com elementos incompletos. O formato **IFCZIP** oferece compressão e compatibilidade entre plataformas.

AULA 2 • PARTE 4

Ajuste de erros no IFC

00:26

O professor retoma o trabalho com arquivos **IFC**, ressaltando a diferença entre testar em modelos pequenos e grandes. Ele mostra como lidar com erros ao carregar arquivos, utilizando condições de existência para evitar falhas ao rodar listas de materiais e gerar tabelas. Introduz a lógica condicional no código para controlar a execução, assegurando que apenas arquivos válidos avancem no processamento.

O uso de condicionais garante integridade no processamento de **IFC**, evitando erros de indexação e falhas de carregamento. O princípio é verificar se o objeto existe antes de manipulá-lo.

05:29

Renomeação e classificação de paredes

É apresentada a função de **renomear paredes** dentro do IFC. O professor demonstra como listar paredes, identificar suas partes e distinguir entre **paredes externas** e **internas**. Para isso, utiliza a relação de decomposição (IsDecomposedBy) e analisa camadas de cada parede, como acabamento externo, acabamento interno e alvenaria. A lógica construída permite corrigir nomenclaturas genéricas e duplicadas, automatizando a classificação com base em critérios técnicos. O código é estruturado para percorrer todas as paredes, tratando casos sem decomposição, como rodapés.

A decomposição no **IFC** estabelece hierarquia entre elementos e subpartes. O reconhecimento de camadas permite padronizar a classificação e melhorar a organização semântica dos modelos.

Exportação e ciclo de modificação

22:58

Concluída a classificação, o professor explica como **salvar o IFC modificado**. Ele mostra a implementação de um botão de **download** que transforma os dados em bytes e gera um novo arquivo IFC editado. O ciclo descrito envolve abrir, modificar, validar, exportar e reutilizar em softwares BIM, com compatibilidade em ferramentas como o **Libre**. Também apresenta a possibilidade de integração com o **GitHub** e a limitação de plataformas gratuitas como o Streamlit, que aceitam apenas arquivos de até 200 MB.

Na sequência, o professor compartilha sua trajetória profissional: engenheiro civil que iniciou com **BIM** em 2011 e percebeu a necessidade de **automação** para superar tarefas repetitivas e onerosas, como simulações 4D. Destaca que a programação foi incorporada para reduzir custos, erros e tempo de execução, reforçando a importância da integração entre engenharia e código.

A exportação de IFC garante **interoperabilidade** e permite o compartilhamento confiável de modelos. A automação surge como resposta à complexidade e à morosidade de processos manuais, consolidando a aplicação da programação no BIM.

EXERCÍCIO DE FIXAÇÃO

Qual comando executa o arquivo principal no Streamlit?

- python app.py
- streamlit run app/app.py
- run app.py
- flask run app

EXERCÍCIO DE FIXAÇÃO

Qual estrutura em Python associa material às áreas?

- Lista
- Tupla
- Dicionário
- Conjunto

EXERCÍCIO DE FIXAÇÃO

Por que usar try/except no processamento de IFC?

- Melhorar desempenho
- Evitar erros de atributos ausentes
- Corrigir nomes duplicados
- Aumentar velocidade de upload

EXERCÍCIO DE FIXAÇÃO

Qual relação IFC diferencia partes de paredes?

- IsDefinedBy
- IsDecomposedBy
- HasProperties
- RelAggregates

Respostas desta página: alternativas 2, 3, 2, 2

AULA 3 • PARTE 1

Configuração do ambiente Python

00:58

O professor apresenta a proposta da unidade: configurar o ambiente de programação em **Python**. São discutidas alternativas de instalação local e o uso de **computação em nuvem,** destacando editores como **VSCode** e **Jupyter Notebook.** Ressalta-se o papel do **Linux** e de seu terminal como padrão para instalação e execução de bibliotecas, além da possibilidade de integração via **WSL** no Windows. O objetivo é garantir flexibilidade no acesso e padronização do ambiente.

O **Python** exige editor de texto e terminal configurado. O **Linux** facilita a padronização de comandos. O **WSL** permite rodar Linux dentro do Windows sem dual boot.

08:48

Instalação, pacotes e privacidade

São explicados os papéis do **APT** e do **pip** como instaladores, respectivamente de programas e bibliotecas. Abordam-se práticas comuns de cursos e tutoriais, além de alternativas de ambientes em nuvem como **Google Colab** e **Microsoft Guide,** que já possuem Python instalado. Introduz-se o uso do **Git** e do **GitHub** para versionamento e colaboração, ressaltando a importância da sincronização de pastas em equipe. Em seguida, compara-se o trabalho em computador local, que exige **backup**, com o uso da nuvem, que garante redundância, mas traz riscos de **exposição de dados.** Destaca-se a necessidade de configurar visibilidade e usar criptografia para preservar a privacidade.

O **APT** instala programas e o **pip** gerencia bibliotecas. O **Git** versiona e sincroniza código. O local dá autonomia, mas exige **backup**; a **nuvem** traz redundância, mas expõe dados.

23:11

Bibliotecas Python e execução prática

São apresentadas bibliotecas essenciais: **ifcopenshell** para manipulação de arquivos **IFC, pandas** para análise tabular e **matplotlib** para gráficos. Explica-se a possibilidade de carregar múltiplas bibliotecas via arquivos de requisitos, garantindo automação. Demonstra-se a abertura de um **arquivo IFC de teste** e a execução de um **primeiro script,** que identifica elementos como paredes A, B e C. A proposta é mostrar como tarefas comuns em planilhas eletrônicas podem ser reproduzidas e ampliadas via **programação em Python.**

O **ifcopenshell** manipula arquivos **IFC**. O **pandas** organiza e analisa dados. O **matplotlib** gera gráficos de visualização.

AULA 3 • PARTE 2

Introdução aos scripts em Python

00:26

Apresenta o uso de **scripts em Python** para **automatizar tarefas** no IFC. Defende começar com **roteiros pequenos** e expandir conforme a necessidade. O Python é mostrado como linguagem acessível para **profissionais não programadores,** útil para **extrair dados** e apoiar a **tomada de decisão.**

A automação **reduz erros humanos, garante regularidade** e **aumenta a produtividade.** O Python é popular por sua simplicidade em comparação a linguagens como C.

05:00

Interoperabilidade e formato IFC

Explica a necessidade de **interoperabilidade** entre softwares como **Revit** e **AutoCAD**. O IFC, padronizado pela ISO e aberto pela BuildingSmart, permite troca de dados entre plataformas. Destaca o conceito de **OpenBIM** e o uso de **OpenShell** para manipulação de arquivos fora de sistemas proprietários.

O **IFC** padroniza entidades como paredes e portas, evitando terminologias divergentes. O **OpenBIM** amplia o acesso a arquivos e favorece o código aberto.

Contagem e exportação de elementos

13:11

Demonstra um script que conta paredes, portas e janelas em arquivos IFC. Mostra a diferença entre métodos manuais e automáticos. Em seguida, apresenta a exportação para **CSV**, permitindo integração com **planilhas** e **relatórios**.

A biblioteca CSV no Python viabiliza **exportação estruturada, reduz tempo** e **custo de tarefas.**

18:04

Escalabilidade e aplicações avançadas

Mostra a aplicação em projetos reais, com milhares ou milhões de entidades. O Python suporta grandes volumes, enquanto planilhas têm limitações. Explica o uso de **propriedades personalizadas, anotações colaborativas** e o **acompanhamento de evolução de obras.** Apresenta o modelo Duplex como exemplo prático.

As propriedades ampliam a **flexibilidade** do IFC. A **escalabilidade** do Python assegura **desempenho em grandes projetos** e apoia decisões com **dados confiáveis.**

AULA 3 • PARTE 3

Uso de memória e Generator

00:26

O professor mostra que copiar listas para o **Pandas** aumenta o uso de memória. Explica que arquivos grandes podem triplicar o consumo. Introduz o **Generator**, que envia dados direto ao dataframe, sem lista intermediária.

O **Generator** cria elementos sob demanda, reduzindo memória e acelerando o fluxo de dados.

02:01

Gráficos e bibliotecas de visualização

Mostra o uso de gráficos para comunicar resultados. Utiliza **Counter** e **Matplotlib** para gerar contagens em setores. Destaca que o **Pandas** simplifica essa operação e pode usar dados exportados em CSV.

O **Counter** cria dicionários de frequência. O **Matplotlib** gera gráficos básicos e o **Pandas** integra análise tabular e visualização.

Geração automática de documentos

10:17

Explica a criação de relatórios automáticos com **python-docx.** Mostra como gerar documentos **.docx** com títulos, listas e dados extraídos do IFC ou CSV. Destaca a automação de propostas e inventários.

O formato **.docx** é padrão aberto. A biblioteca **python-docx** replica recursos comuns do Word, como tabelas e estilos.

21:35

Exercícios e prática em programação

Conclui ressaltando a importância da prática. Recomenda resolver exercícios no **Beecrowd**, com feedback automático, para consolidar uso de listas, laços, pandas e bibliotecas.

A **programação competitiva** reforça lógica e cria memória de código por meio de desafios regulares.

AULA 3 • PARTE 4

Scripts e manutenção no IFC

00:26

O professor introduz a Unidade 3 destacando que os estudantes já criam **scripts**. Ele mostra a importância de revisar e ajustar códigos para facilitar a **manutenção**. Apresenta ainda a possibilidade de **gravar no IFC,** permitindo anotações em cópias do modelo sem alterar a **fonte original.**

O **IFC** suporta **anotações** adicionais em cópias. Essas alterações não substituem os dados originais do **contratante** e devem ser reaplicadas a cada nova importação do modelo.

02:07

Funções de filtro e comando Wild

Explica-se o padrão comum nos **scripts: carregar, filtrar, extrair e gerar saídas como CSV ou relatórios.** Apresenta a função filter, que seleciona elementos por tipo e nome, e o comando Wild, usado como generator para reduzir consumo de memória.

O uso de funções evita repetição de código e organiza a programação. **Iteradores** como **Wild** permitem processar elementos de forma incremental.

Anotações e dimensões do OpenBIM

08:02

Demonstra-se como criar **property sets** para registrar informações por dimensão: 4D (tempo), 5D (custo), 6D (sustentabilidade), 7D (manutenção) e 8D (segurança). Exemplos incluem fases de alvenaria, custos de portas, eficiência energética, inspeção de lajes e áreas de risco.

Property sets agrupam **atributos** de elementos e seguem **convenções de nomenclatura.** Permitem consultas por tempo, custo ou segurança, apoiando análises além do 3D.

31:42

Comparação e parametrização

Apresenta a comparação entre **arquivos IFC** com o comando **diff** no **Linux**, mostrando alterações. Explica o uso dos scripts **Mostra Paredes** e **Mostra Duplex**, que permitem **parametrizar** o arquivo de entrada. Ressalta o uso de **prints** para **depuração** e a importância de **testes automatizados** em códigos maiores.

A **parametrização** amplia a reutilização dos scripts. O controle via **property sets** evita conflitos de nomes. **Depuração** e **testes** garantem robustez e confiabilidade.

EXERCÍCIO DE FIXAÇÃO

Qual é a principal função do WSL no contexto do uso do Python?

- Criar gráficos interativos
- Substituir o Git
- Rodar Linux dentro do Windows
- Gerar relatórios automáticos

EXERCÍCIO DE FIXAÇÃO

Qual biblioteca é usada para manipular arquivos IFC?

- Pandas
- IFC OpenShell
- Matplotlib
- Numpy

EXERCÍCIO DE FIXAÇÃO

Qual biblioteca do Python permite criar documentos .docx?

- Matplotlib
- Pandas
- Python-docx
- Csv

EXERCÍCIO DE FIXAÇÃO

Qual comando no Linux compara diferenças entre arquivos IFC?

- Grep
- Diff
- Cat
- Ls

Respostas desta página: alternativas 3, 2, 3, 2

Resumo da disciplina

Veja, nesta página, um resumo dos principais conhecimentos adquiridos ao longo da disciplina.

AULA 1

O Open BIM assegura interoperabilidade entre softwares de construção.

As variáveis booleanas permitem expressar condições lógicas em Python.

O IFC é estruturado em camadas hierárquicas de dados.

AULA 2

O Streamlit cria interfaces web interativas para Python.

A filtragem por material garante padronização na análise de modelos.

O uso de dataframes organiza materiais e áreas em formato tabular.

AULA 3

O Linux padroniza comandos e simplifica a configuração do ambiente.

A automação em Python reduz erros e aumenta a produtividade.

O Generator otimiza memória em grandes arquivos de dados.

Avaliação

Veja as instruções para realizar a avaliação da disciplina.

Já está disponível o teste online da disciplina. O prazo para realização é de **dois meses a partir da data de lançamento das aulas**.

Lembre-se que cada disciplina possui uma avaliação online.
A nota mínima para aprovação é 6.

Fique tranquilo! Caso você perca o prazo do teste online, ficará aberto o teste de recuperação, que pode ser realizado até o final do seu curso. A única diferença é que a nota máxima atribuída na recuperação é 8.

PUCRS online

uol edtech